The Expanding Attack Surface of Agentic AI: Vulnerabilities, Exploitation Vectors, Defenses, and Benchmarks

Seyedakbar Mostafavi

***Abstract*—The integration of Large Language Models (LLMs) with iterative planning routines, persistent state repositories, heterogeneous external tools, and multi-agent coordination frameworks has driven the transition from static conversational systems to autonomous Agentic Artificial Intelligence (AI). While this architecture unlocks broad autonomous capabilities, it concurrently dismantles traditional software and security perimeters. Agentic systems grant probabilistic models direct effectuation authority over digital and physical environments, collapsing the conventional distinction between untrusted data and executable control logic. This paper presents a systematic survey and rigorous taxonomical framework of the attack surface, exploitation vectors, defensive architectures, and empirical evaluation testbeds governing agentic AI.**

We formalize an end-to-end reference architecture across the perception-cognition-action-reflection lifecycle, systematically dissecting the expanded Trusted Computing Base (TCB) and delineating fragile trust boundaries across single-agent, hierarchical, and decentralized multi-agent topologies. Expanding upon foundational systematizations, we establish an analytical taxonomy of attacks spanning prompt-level manipulations, Retrieval-Augmented Generation (RAG) and embedding poisoning, memory and reflection corruption, tool shadowing and Remote Code Execution (RCE), Model Context Protocol (MCP) vulnerabilities, and multi-agent Byzantine consensus subversion. Furthermore, we critically analyze defense-in-depth mechanisms, contrasting empirical input/output guardrails with formal policy verification, capability-based sandboxing, and runtime invariant enforcement. Finally, we provide a unified comparative matrix of contemporary security benchmarks, examine emerging governance mandates, and articulate an agenda of open research challenges required to engineer secure, resilient, and verifiable autonomous systems.

***Index Terms*—Agentic AI, Large Language Model (LLM) Security, Attack Surface, Tool Execution Security, Multi-Agent Systems (MAS), Model Context Protocol (MCP), Retrieval-Augmented Generation (RAG) Poisoning, Indirect Prompt Injection, Defense-in-Depth, Security Benchmarking.**

I. Introduction

THE development of foundation models and distributed intelligent systems has accelerated the transition in artificial intelligence from passive sequence-to-sequence text generation toward autonomous, trustworthy *Agentic AI* architectures [1]–[7]. Contemporary agentic architectures synthesize Large Language Models (LLMs) with algorithmic planning engines, long-term state repositories, external knowledge retrievers, and execution interfaces (such as tools, software APIs, and operating system shells) [8]. By executing iterative loops of perception, cognitive reasoning, tool invocation, and environment feedback, these agents perform multi-horizon workflows across software engineering, autonomous data analysis, enterprise resource planning, and cyber operations [9], [10].

Despite these operational capabilities, the transition toward autonomous agency introduces fundamental dependability and security vulnerabilities [11], [12]. In traditional computational systems, execution environments maintain strict hardware- and kernel-enforced isolation between untrusted data streams and executable control logic [6]. In contrast, LLM-based agentic architectures process natural language instructions, operational guidelines, retrieved third-party content, conversation history, and tool definitions within a single, shared semantic context space, creating vulnerability to adversarial manipulations [13], [14]. Consequently, probabilistic foundation models cannot deterministically segregate authoritative system instructions from adversarial payloads embedded in external data [15]–[17].

When an LLM is granted effector privileges (such as executing shell scripts, issuing HTTP requests, mutating database records, or initiating financial transactions), the failure to isolate code from data creates severe systemic risks [6], [9]. Empirical investigations demonstrate that prompt manipulations can trigger Remote Code Execution (RCE) [9], targeted data poisoning can subvert Retrieval-Augmented Generation (RAG) pipelines [18]–[20], and malicious cross-agent signaling can provoke cascading failures across multi-agent systems [5], [21]–[23].

A. Foundations and Limitations of Prior Systematizations

In a foundational Systematization of Knowledge (SoK), Dehghantanha and Homayoun [6] mapped the primary trust boundaries and attack surfaces of tool-augmented LLM systems. Their work codified five canonical threat paths (P1–P5), defined seven attacker-aware evaluation metrics (such as Unsafe Action Rate and Privilege Escalation Distance), and synthesized early evidence from the 2023–2025 research epoch. Other contemporary overviews examine specific facets of generative security and trustworthy distributed intelligence: Min *et al.* [1] reviewed foundational LLM developments; Wei and Liu [2] analyzed robustness, privacy, and governance in distributed AI; Chander *et al.* [3] surveyed explainability and robustness in trustworthy AI; Goyal *et al.* [4] categorized adversarial defenses in NLP; and Standen *et al.* [5] investigated adversarial machine learning in multi-agent environments; alongside agent vulnerability surveys by Deng *et al.* [7]; high-level threat

S. Mostafavi is with the Department of Computer Engineering, Yazd University, Yazd 89195-741, Iran (e-mail: a.mostafavi@yazd.ac.ir).

models by Narajala and Narayan [8]; trust, risk, and security governance (TRiSM) by Raza *et al.* [24]; and coordination challenges by Schroeder de Witt *et al.* [25].

However, existing literature exhibits five structural limitations:

1) **Omission of Multimodal and Computer-Use Frontiers:** Prior surveys focus almost exclusively on text-based function calling, leaving graphical user interface (GUI) automation, visual prompt injection, and OS-level Computer-Use architectures underexplored [21], [26].
2) **Absence of Standardized Tool Protocol Analysis:** The rapid standardization of agent interfaces, exemplified by the Model Context Protocol (MCP) [27] and OpenAI function calling standards, introduces novel host-client-server trust dynamics and tool shadowing risks that have not yet been systematically formalized.
3) **Superficial Multi-Agent System (MAS) Topologies:** While initial studies acknowledge inter-agent prompt propagation [21], [25], they do not systematically categorize vulnerabilities arising from heterogeneous coordination topologies (such as hierarchical orchestrators, peer-to-peer swarms, blackboard architectures, and consensus voting mechanisms).
4) **Bifurcation of Empirical Guardrails and Formal Verification:** The literature remains divided between heuristic, empirical prompt filters [28], [29] and formal systems security principles [15]. A unified framework integrating capability-based access control (CBAC), temporal logic plan verification, and runtime invariant enforcement is conspicuously lacking.
5) **Fragmented Benchmarking Testbeds:** Current security benchmarks (such as AgentDojo [30], InjecAgent [31], and ToolEmu [32]) operate in disjoint evaluation silos with divergent threat assumptions, hindering reproducible cross-paradigm comparisons.

B. Contributions and Organization

To bridge these gaps, this paper establishes a systematic survey that broadens and formalizes the security analysis of agentic AI. The primary contributions of this work are as follows:

- **Comprehensive Architectural Decomposition:** We formulate an end-to-end reference architecture encompassing the full perception-cognition-action-reflection lifecycle, systematically mapping the expanded Trusted Computing Base (TCB) across single-agent, hierarchical, and decentralized coordination topologies.
- **Granular Attack Surface Taxonomy:** We establish an analytical taxonomy of exploitation vectors, decomposing attack goals (G1–G7) into concrete micro-primitives, including memory sleep-cycle corruption, semantic cache poisoning, reflection hijacking, tool parameter smuggling, and Insecure Direct Object Reference (IDOR) vulnerabilities in retrieval embeddings.
- **Frontier Analysis of Protocols and Modalities:** We provide a systematic security treatment of the Model Context Protocol (MCP) and investigate multimodal attack surfaces, including visual typographic injection and OS-level GUI manipulation.
- **Multi-Agent Threat Modeling:** We categorize topology-specific vulnerabilities in Multi-Agent Systems (MAS), modeling Byzantine consensus degradation, collective hallucination loops, and self-replicating prompt worms.
- **Holistic Defense-in-Depth Framework:** We formulate a tiered defensive taxonomy that bridges empirical guardrails, structured query enforcement, capability-based sandboxing, and formal contract-based plan verification.
- **Unified Benchmark and Governance Synthesis:** We present a comparative matrix of contemporary agent security benchmarks and map technical vulnerabilities to emerging international governance frameworks (OWASP GenAI Top 10, MITRE ATLAS, NIST AI RMF, and the EU AI Act).

The remainder of this paper is structured as follows. Section II introduces the agentic AI reference architecture and operational lifecycle. Section III formalizes the threat model, adversary profiles, and causal threat graph. Section IV establishes the unified taxonomy of attack vectors and multi-stage kill chains. Section V examines multimodal GUI and protocol-level vulnerabilities. Section VI investigates multi-agent coordination risks. Section VII synthesizes multi-layer defensive paradigms. Section VIII evaluates security metrics and benchmarking frameworks. Section IX analyzes governance and standards. Section X articulates open research problems, and Section XI concludes the paper.

II. Agentic AI Reference Architecture and Operational Lifecycle

To establish a rigorous foundation for vulnerability analysis, we formalize the structural components, execution pipelines, and operational paradigms that characterize modern Agentic AI systems.

A. Formal System Model

We formalize an autonomous agentic system as a sextuple

$$\mathcal{A} = \langle \mathcal{M}, \mathcal{P}, \mathcal{S}, \mathcal{T}, \mathcal{E}, \Omega \rangle, \tag{1}$$

where $\mathcal{M}$ denotes the core foundation model (e.g., an autoregressive Large Language Model or Vision-Language Model) parameterized by pre-trained weights θ, responsible for semantic parsing, natural language generation, and zero-shot reasoning. Table I details the mathematical symbols and operational nomenclature adopted throughout this paper.

The formal constituents of tuple $\mathcal{A}$ are structured as follows:

- $\mathcal{P}$ represents the deliberative planning engine governing decomposition, tool selection heuristics, and policy execution (e.g., ReAct, Reflexion, Tree-of-Thoughts, or static workflow state machines).
- $\mathcal{S} = \langle \mathcal{S}_{\text{work}}, \mathcal{S}_{\text{ep}}, \mathcal{S}_{\text{sem}} \rangle$ designates the hierarchical memory architecture, comprising active working context ($\mathcal{S}_{\text{work}}$), short-term episodic scratchpads ($\mathcal{S}_{\text{ep}}$), and long-term semantic knowledge bases ($\mathcal{S}_{\text{sem}}$) indexed via vector embeddings.

TABLE I
SUMMARY OF MAIN NOTATIONS AND MATHEMATICAL SYMBOLS.

Notation	Formal Definition and Semantic Description
$\mathcal{A}$	Autonomous agent tuple $\langle \mathcal{M}, \mathcal{P}, \mathcal{S}, \mathcal{T}, \mathcal{E}, \Omega \rangle$
$\mathcal{M}, \theta$	Foundation model core and neural parameter weights
$\mathcal{P}$	Deliberative planning policy and decomposition engine
$\mathcal{S}_{\text{work}}$	Active working memory and conversational context window
$\mathcal{S}_{\text{ep}}, \mathcal{S}_{\text{sem}}$	Short-term episodic scratchpad and long-term semantic RAG store
$\mathcal{T}, t_i$	Tool registry and individual tool capability tuple
$\mathcal{E}$	External execution environment (OS, network, cloud, peers)
Ω	Orchestration substrate, schema validator, and security broker
$\mathcal{G}_{\text{threat}}$	Causal threat graph $(\mathcal{V}, \mathcal{E}, \mathcal{K})$ modeling exploit progression
$\mathcal{V}_{\text{source}}, \mathcal{V}_{\text{asset}}$	Subsets of untrusted ingress sources and protected assets
$\mathcal{K}(e)$	Defensive security controls capable of severing transition edge e
$\mathcal{A}_{\text{adv}}$	Six-tier adversary set $\{\mathcal{A}_{\text{ext}}, \mathcal{A}_{\text{cont}}, \mathcal{A}_{\text{tool}}, \mathcal{A}_{\text{peer}}, \mathcal{A}_{\text{sc}}, \mathcal{A}_{\text{ins}}\}$
$N(t), \beta, k$	Infected agent count, transmission rate, and peer degree in MAS
UAR, PAR	Unsafe Action Rate and Policy Adherence Rate
PED	Privilege Escalation Distance (shortest path in $\mathcal{G}_{\text{threat}}$)
TTC, PHL	Time-to-Contain and Patch Half-Life
RRS$(\mathcal{B})$	Retrieval Risk Score over retrieved document bundle $\mathcal{B}$
OORAR	Out-of-Role Action Rate under operational contract $\mathcal{C}$
CES$_T(\pi)$	Cost-Exploit Susceptibility over execution horizon T
$\mathcal{I}(s), \delta(s, a)$	Safety invariant predicate and predicted post-action state

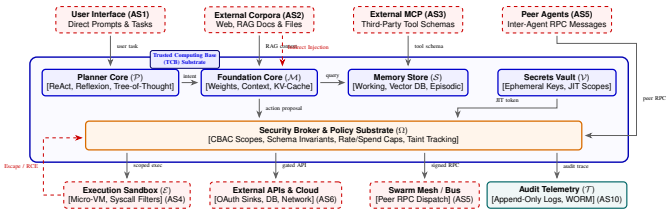

Fig. 1. Agentic AI reference architecture and Trusted Computing Base (TCB) delineation. Blue boxes denote verified TCB modules ($\mathcal{P}, \mathcal{M}, \mathcal{S}, \Omega$), while dashed red containers highlight untrusted ingress and execution surfaces. Gray arrows trace capability flows mediated by security broker Ω; dashed red vectors indicate prompt injection and sandbox escape paths.

- $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$ denotes the tool registry, where each tool $t_i = \langle \text{ID}_i, \text{Schema}_i, \text{Perm}_i, \text{Endpoint}_i \rangle$ defines a concrete external capability (e.g., code interpreter, filesystem access, web client, or cloud API) governed by typed input/output schemas.
- $\mathcal{E}$ represents the external execution environment, encompassing local operating system primitives, enterprise database instances, network perimeters, and peer agent endpoints deployed across real-world enterprise architectures [33]–[35].
- Ω defines the orchestration and security enforcement substrate, comprising prompt templates, schema validators, capability brokers, audit loggers, and human-in-the-loop (HITL) authorization gates.

B. The Perception-Cognition-Action-Reflection Lifecycle

The core operational cycle of an agent unfolds across five discrete phases, iterated until a terminal objective is satisfied, a computation budget is exhausted, or a policy violation triggers an abort:

1) **Ingestion and Perception:** The agent ingests user intent $u \in \mathcal{U}$ alongside retrieved context $c_{\text{ret}} \sim \mathcal{S}_{\text{sem}}$ and sensory telemetry (e.g., DOM trees, terminal output, or GUI screenshots).
2) **Deliberative Cognition and Planning:** Conditioned on memory state $\mathcal{S}$ and system policies, the planner $\mathcal{P}$ generates an intermediate chain-of-thought rationale τ_t and synthesizes a candidate action $a_t \in \mathcal{T} \cup \{\text{TERMINATE}\}$.
3) **Brokering and Validation:** The security broker Ω intercepts candidate action $a_t = \langle t_k, \mathbf{x}_k \rangle$, validating parameter bundle $\mathbf{x}_k$ against schema Schema$_k$ and verifying execution permissions Perm$_k$.
4) **Environment Execution:** Upon clearance, the tool endpoint Endpoint$_k$ executes action a_t within environment $\mathcal{E}$, yielding raw observation $o_t \in \mathcal{O}$.
5) **State Update and Reflection:** The observation o_t is parsed, evaluated against safety criteria, and appended to working memory $\mathcal{S}_{\text{work}}$, triggering potential self-correction or long-term memory consolidation in $\mathcal{S}_{\text{sem}}$.

C. Architectural Paradigms and Coordination Topologies

Modern agentic deployments exhibit diverse coordination topologies that shape the perimeter and propagation dynamics of security threats:

- **Autonomous Single-Agent Systems:** A monolithic agent executes end-to-end task loops via dedicated tool interfaces. While structurally straightforward, single-agent architectures exhibit high vulnerability to linear kill-chain progression, as the compromise of the central model directly yields full effector privileges [6], [9].
- **Hierarchical Orchestrator-Worker Networks:** A centralized orchestrator decomposes high-level goals and delegates sub-tasks to specialized worker agents possessing isolated tool subsets. Security vulnerabilities in this paradigm center on delegation abuse, privilege escalation via sub-agent prompt spoofing, and orchestrator hijacking [8].
- **Decentralized Swarms and Consensus Meshes:** Autonomous agents interact peer-to-peer over a shared communication bus or blackboard architecture without a single coordinator. Consensus mechanisms (e.g., majority voting, iterative debate, or token-weighted aggregation) are employed to align decisions. Open communication buses introduce susceptibility to Byzantine corruption and prompt-based epidemic spreading [21], [23], [25].
- **Grounded GUI and Computer-Use Agents:** Multimodal architectures interact directly with desktop and operating system interfaces via mouse clicks, keystrokes, and screenshot parsing. These agents bridge semantic understanding with native OS execution, expanding the attack surface beyond constrained API boundaries [26].

D. Standardization Protocols: The Model Context Protocol (MCP)

A foundational architectural advancement in modern agentic ecosystems is the standardization of client-host-server tool protocols, most prominently Anthropic's *Model Context Protocol*

(MCP) [27]. MCP decomposes agent integration into three distinct architectural tiers:

1) **MCP Host:** The host application (e.g., an IDE, desktop assistant, or workflow runner) that manages model execution, user authorization, and agent lifecycles.
2) **MCP Client:** The intermediary adapter embedded within the host that negotiates connections, maintains session state, and translates LLM tool requests into protocol calls.
3) **MCP Server:** Independent, standalone lightweight services exposing distinct capabilities categorized into: (i) *Resources* (file and database contexts), (ii) *Prompts* (pre-packaged workflow templates), and (iii) *Tools* (executable functions with side effects).

While MCP establishes clean decoupling between foundational models and external capability providers, this client-host-server separation creates protocol-level boundaries that, if left unhardened, introduce severe serialization, impersonation, and tool-shadowing vulnerabilities, as analyzed in Section V.

III. Threat Modeling, Trust Boundaries, and the Causal Threat Graph

Securing agentic AI requires establishing formal threat models that delineate the system's Trusted Computing Base (TCB), identify adversary capabilities, and map causal attack propagation routes.

A. Delineating the Trusted Computing Base (TCB)

In classical security engineering, the TCB denotes the minimal set of hardware, firmware, and software elements whose correct enforcement is strictly required to uphold system security properties. In agentic architectures, defining the TCB is complicated by the non-deterministic, probabilistic nature of foundation models [6], [36].

We classify the architectural components into trusted vs. untrusted perimeters:

- **Inside the TCB (Core Security Substrate):**
  1) *Foundation Model Core ($\mathcal{M}$):* The base model weights and inherent safety alignment parameters, assumed free from upstream backdoors unless modeled under supply-chain compromise.
  2) *Security Broker and Policy Engine (Ω):* The deterministic execution substrate responsible for strict JSON-schema validation, access policy enforcement, capability tokens, and rate limits.
  3) *Secrets Vault and Token Dispenser:* The isolated hardware or software credential store that issues ephemeral, scoped tokens to tool effectors without disclosing master keys to context memory.
  4) *Verified Memory and Storage Substrate:* The append-only, tamper-evident audit loggers, cryptographic hash stores, and access-controlled enterprise knowledge repositories.
- **Outside the TCB (Hostile and Untrusted Boundaries):**
  1) *User and Ingress Channels:* Direct prompts, chat interfaces, and webhook triggers.
  2) *External Knowledge Corpora:* Web pages, public document collections, forum scrapes, and third-party databases ingested during RAG operations.
  3) *Tool and Effector Endpoints:* External shell interpreters, container sandboxes, third-party MCP servers, and cloud service endpoints.
  4) *Inter-Agent Communication Buses:* Network channels and shared memory workspaces utilized by peer agents in distributed multi-agent topologies.

B. Formal Adversary Taxonomy

Expanding upon the foundational adversary classes introduced by Dehghantanha and Homayoun [6] and threat modeling methodologies [36], we formalize a six-tier adversary model $\mathcal{A}_{\text{adv}} = \{\mathcal{A}_{\text{ext}}, \mathcal{A}_{\text{cont}}, \mathcal{A}_{\text{tool}}, \mathcal{A}_{\text{peer}}, \mathcal{A}_{\text{sc}}, \mathcal{A}_{\text{ins}}\}$:

1) **External Unprivileged Attacker ($\mathcal{A}_{\text{ext}}$):** Interacts with the agent through primary user interfaces without authenticated administrative rights. Capabilities include direct prompt injection, execution trigger manipulation [37], jailbreak crafting (e.g., multi-turn Crescendo attacks [38]), and adversarial queries designed to provoke policy drift or data extraction [39].
2) **Malicious Content Provider ($\mathcal{A}_{\text{cont}}$):** Exerts indirect control over passive digital assets ingested by the agent (e.g., hosting attacker-controlled web pages, publishing trojaned PDFs, or modifying public code repositories). The attacker never directly interfaces with the agent; instead, malicious directives are ingested via web browsing, file parsing, or RAG indexing [13], [14], [18].
3) **Rogue Tool / MCP Provider ($\mathcal{A}_{\text{tool}}$):** Operates or compromises a third-party plugin, API service, or MCP server integrated into the agent's tool registry $\mathcal{T}$. The adversary can forge deceptive schema specifications, manipulate tool response payloads, trigger server-side request forgery (SSRF), or intentionally induce resource loops [9].
4) **Byzantine Peer Agent ($\mathcal{A}_{\text{peer}}$):** Possesses legitimate participant status within a decentralized or collaborative multi-agent network. The adversary injects malicious inter-agent messages, forges coordination telemetry, manipulates consensus voting, or propagates self-replicating adversarial instructions [21], [23], [25].
5) **Supply-Chain Compromiser ($\mathcal{A}_{\text{sc}}$):** Targets upstream assets prior to deployment, such as poisoning open-source model checkpoints, corrupting embedding generators, implanting backdoors into foundational agent libraries (e.g., LangChain, AutoGen), or tampering with package dependency mirrors [9], [10].
6) **Insider / Compromised Administrator ($\mathcal{A}_{\text{ins}}$):** Holds authenticated credentials or developer access to system prompts, vector databases, or deployment infrastructure. Misconfigurations, credential leakage, or deliberate sabotage can bypass external perimeters entirely [40].

C. Assets and Security Properties

The assets at risk in agentic systems span four foundational tiers:

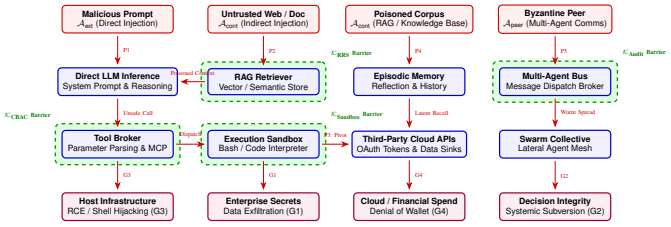

Fig. 2. Causal threat graph of exploit propagation paths (P1–P5) to target assets (G1–G4). Red nodes indicate adversary footholds, blue nodes show internal execution components, and purple nodes denote target assets. Green dashed contours mark interdiction barriers that sever the kill chain.

- **Data Confidentiality:** Proprietary system prompts, retrieved tenant documents, user session records, and embedded cryptographic credentials.
- **Behavioral and Decision Integrity:** Ensuring the agent adheres strictly to developer-defined operational policies, executes valid tool invocations, and produces factually grounded outputs free from external manipulation.
- **System and Financial Availability:** Protecting underlying compute infrastructure, preventing infinite execution loops, and avoiding "Denial of Cash" (DoC) attacks resulting from unbounded API calls or serverless resource exhaustion [41], [42].
- **Accountability and Forensics:** Preserving non-repudiation and tamper-evident audit trails across automated planning and effector traces.

D. The Causal Threat Graph Formulation

To mathematically analyze how initial attack footholds propagate into catastrophic system compromises, we formalize the system's risk structure as a *Causal Threat Graph* $\mathcal{G}_{\text{threat}} = (\mathcal{V}, \mathcal{E}, \mathcal{K})$, defined as a directed multigraph where:

- $\mathcal{V} = \mathcal{V}_{\text{source}} \cup \mathcal{V}_{\text{core}} \cup \mathcal{V}_{\text{effect}} \cup \mathcal{V}_{\text{asset}}$ represents the disjoint union of untrusted input sources, agent pipeline stages, effector sinks, and protected enterprise assets.
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denotes directed causal transitions, where directed edge $e = (v_i, v_j) \in \mathcal{E}$ represents the capability of an exploit at component v_i to directly influence or compromise state at v_j.
- $\mathcal{K} : \mathcal{E} \to \mathcal{P}(\mathcal{D})$ maps each transition edge to the set of defensive security controls $\mathcal{D}$ (e.g., input sanitization, schema validation, isolated sandboxing, or egress filtering) capable of severing the causal link.

Formally, an end-to-end exploit constitutes a directed path $\pi = \langle v_0, v_1, \ldots, v_m \rangle$ in $\mathcal{G}_{\text{threat}}$ such that $v_0 \in \mathcal{V}_{\text{source}}$ and $v_m \in \mathcal{V}_{\text{asset}}$. As illustrated in Fig. 2, a system is provably resilient against exploit path π if and only if there exists at least one edge $(v_i, v_{i+1}) \in \pi$ whose associated defensive controls $\mathcal{K}(v_i, v_{i+1})$ are actively and deterministically enforced.

IV. Comprehensive Taxonomy of Attack Vectors and Multi-Step Kill Chains

To systematically dissect how adversaries exploit agentic AI, we establish a multi-dimensional taxonomy categorizing attack objectives, pipeline vectors, and multi-stage exploit pathways. Table II synthesizes these vectors alongside their operational preconditions, path mappings, and standardizations.

A. Categorization of Attacker Objectives

Attacker motivations in agentic environments extend beyond conversational misuse into external computational and operational infrastructure. We structure attacker goals into *Enablers* (which expand operational capabilities) and *Terminal Outcomes* (which realize concrete damage):

- **Enablers:**
  - *G3: Privilege Escalation / Unauthorized Invocation:* Upgrading an unprivileged natural language prompt into operating system commands, cloud administrative API calls, or restricted internal tool execution [6], [9].
  - *G6: Persistence and Backdoor Implantation:* Infiltrating long-term vector stores, conversation memory, or configuration prompts to maintain durable control across subsequent sessions [43], [44].
  - *G7: Upstream Supply-Chain Compromise:* Subverting foundational models, third-party libraries, or tool packages before agent deployment [10].
- **Terminal Outcomes:**
  - *G1: Data Exfiltration and Privacy Leakage:* Extracting enterprise databases, proprietary system prompts, or user credentials via out-of-band network channels, DNS tunneling, or disguised tool outputs [13].
  - *G2: Integrity Subversion and Decision Biasing:* Forcing the agent to deviate from true user intent, generating fraudulent summaries, or manipulating automated decision workflows [19].
  - *G4: Resource Abuse and Denial of Service / Cash (DoS/DoC):* Inducing infinite execution loops, pathological recursion, or redundant high-cost API calls to exhaust computational quotas and incur financial penalties [41], [42].
  - *G5: Financial Fraud and Physical Harm:* Executing unauthorized monetary transfers, placing fraudulent commercial orders, or misconfiguring cyber-physical actuators [8], [45].

B. Granular Attack Vectors Across Pipeline Stages

1) Perception and Context Ingress: Adversaries exploit the absence of formal separation between instructional metadata and semantic payload:

- *Direct Prompt Injections and Multi-Turn Jailbreaks:* Direct inputs crafted to override core safety policies. Recent multi-turn techniques, such as the Crescendo attack [38], exploit conversational momentum to guide models into unsafe policy spaces without triggering single-turn semantic filters. In parallel, adversarial perturbations exploit transferability across model boundaries [46]–[48].
- *Indirect Prompt Injection (IPI):* Adversarial directives embedded within untrusted external artifacts (e.g., HTML web pages, PDF documents, and incoming emails) [13], [14]. When the agent retrieves and ingests these artifacts, the model conflates untrusted data with operator instructions, autonomously executing unintended effector commands.
- *Delimiter Collision and Context Stuffing:* Injecting syntax-matching delimiter tokens (e.g., ```` ```json ````, `<system>`,

or XML boundaries) to prematurely close data encapsulation blocks and execute injected commands.

2) Retrieval-Augmented Generation (RAG) and Embeddings: External knowledge retrieval introduces distinct vector-space vulnerabilities:

- *Targeted Corpus Poisoning:* As demonstrated in PoisonedRAG [18], an attacker inserts a minimal number of adversarial documents into the indexing corpus. When benign user queries trigger vector retrieval, the poisoned documents are ranked into top-k context, corrupting the generation output [19].
- *Retrieval Jamming with Blocker Documents:* Shafran *et al.* [20] demonstrated that attackers can inject crafted "blocker documents" that achieve artificially high cosine similarity across broad semantic query spaces, crowding out legitimate knowledge and denying relevant retrieval (Denial of Retrieval).
- *Insecure Direct Object Reference (IDOR) in Vector Retrieval:* In multi-tenant vector databases lacking granular, document-level access-control lists (ACLs), user queries can inadvertently retrieve embeddings belonging to other tenants, enabling cross-tenant data leakage.

3) Memory and Reflection Subsystems: Agentic persistence mechanisms create durable targets:

- *Episodic Memory Poisoning:* Adversaries induce agents to store malicious summaries or poisoned factual associations in long-term memory (e.g., via AgentPoison [43]), establishing persistent backdoors that trigger on specific keywords weeks after initial exposure [44].
- *Reflection and Self-Correction Hijacking:* When agents utilize iterative self-reflection (Reflexion) to critique prior outputs, adversarial observations can mislead the reflection module into false corrections, reversing valid security assertions.

4) Tool Execution and Environment Interaction: The interface between probabilistic reasoning and deterministic OS execution represents a primary attack surface:

- *Parameter Smuggling and Type Confusion:* Attackers exploit weak schema validation to inject unexpected arguments (e.g., newline injection in shell commands, SQL fragments in database queries, or path traversal tokens in file tools) [9], mirroring classical code and query injection patterns [49], [50].
- *Tool Shadowing and Squatting:* In dynamic environments supporting run-time tool discovery, an attacker registers a malicious tool with semantic descriptions nearly identical to a legitimate tool (e.g., naming a rogue tool `fetch_secure_url`), tricking the planner into routing sensitive data through the rogue endpoint.
- *Excessive Agency and Sandbox Escapes:* Insecure container configurations, permissive filesystem mounts, and unrestricted outbound network access allow injected commands to break out of temporary workspaces and compromise host operating systems [9].

C. Formal Multi-Step Exploit Pathways (Kill Chains)

Real-world exploits rarely execute as isolated events; rather, attackers compose multi-step kill chains that traverse trust boundaries:

- **P1: Direct Prompt → Immediate Tool Misuse:** The adversary issues a crafted jailbreak directly commanding the agent to invoke a privileged tool (e.g., executing a reverse shell), achieving immediate privilege escalation (G3).
- **P2: Indirect Ingress → Context → Effector Hijack:** A victim user tasks an autonomous assistant with summarizing a public URL. The URL contains hidden IPI directives instructing the agent to search local files for `id_rsa` and transmit it via HTTP GET to an attacker-controlled endpoint (G1) [13], [14].
- **P3: Cross-Tool Privilege Pivoting:** The attacker abuses an unprivileged tool (e.g., a code interpreter) to stage a malicious file on disk, then induces a secondary tool (e.g., a report generator or email client) to read and dispatch the payload, pivoting across permission domains.
- **P4: Knowledge Poisoning → Latent Retrieval → Action Trigger:** Poisoned documents injected into enterprise documentation lie dormant until an executive user queries the agent. The retrieved text directs the agent to misroute confidential financial approvals (G2, G5) [18], [20].
- **P5: Multi-Agent Cascade (Swarm Epidemics):** A compromised agent transmits a crafted payload across an internal communication bus. Recipient agents interpret the payload as legitimate instructions, propagating the compromise exponentially across the agent collective [21], [25].
- **P6: Human-in-the-Loop Deceptive Justification:** When high-risk actions require human sign-off, an adversarial prompt forces the agent to synthesize a deceptive, highly plausible operational justification (e.g., "Performing critical database maintenance to prevent outage") while masking destructive or exfiltrative underlying parameters, exploiting confirmation fatigue.

V. Frontier Attack Surfaces: Multimodal GUI Agents and the Model Context Protocol (MCP)

As agentic AI expands beyond structured text APIs into multimodal operating system control and standardized tool protocols, novel systemic attack vectors emerge that challenge existing security boundaries.

A. Multimodal GUI and Computer-Use Agent Attack Surfaces

The emergence of computer-use agents (e.g., Anthropic Claude Computer Use and systems evaluated on OSWorld [26]) enables foundation models to interact directly with desktop operating systems via display screenshots, virtual mouse movements, and keyboard keystrokes. This grounded interaction model introduces distinct physical and visual attack surfaces:

1) **Visual Prompt Injection (VPI):** In contrast to plain-text injection, VPI exploits the visual perception pathways of Vision-Language Models (VLMs). Adversaries can

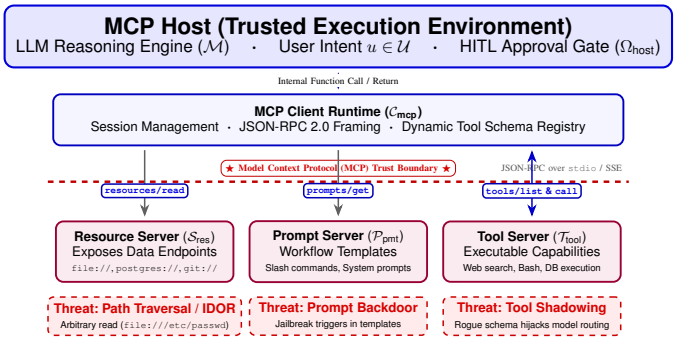

Fig. 3. Model Context Protocol (MCP) architecture and trust boundaries. The Host and Client form the Trusted Computing Base (TCB), communicating via JSON-RPC 2.0 with external, semi-trusted MCP servers across the boundary. Red dashed boxes highlight primary attack surfaces: URI path traversal in resource reads, prompt backdoor injection in workflow templates, and tool shadowing during dynamic capability registration.

embed typographic instructions into rendered web pages, desktop wallpapers, or document images (e.g., micro-fonts, low-contrast text, or steganographic perturbations) [21]. When the agent captures the screen to determine the next UI action, the visual encoder parses the malicious text, steering the agent to click malicious links or execute arbitrary shell commands. Recent browser-resident detection frameworks (e.g., BSM [51]) attempt to intercept such DOM manipulations and API abuse before rendering.

2) **UI Redressing and Visual Clickjacking:** Malicious websites or applications can render transparent overlays, fake system dialogs, or deceptive confirmation buttons designed to mislead the agent's spatial grounding algorithms. An agent intending to click a "Cancel" button may be visually induced to click an underlying "Grant Full Disk Access" prompt.
3) **Operating System State Poisoning:** Unrestricted computer-use agents read system logs, browser histories, and local configuration files. An attacker placing malicious directives in a local terminal buffer or system error log can achieve delayed compromise whenever the agent inspects system state for troubleshooting.

B. Security Analysis of the Model Context Protocol (MCP)

The Model Context Protocol (MCP) has emerged as an open standard for connecting LLM applications to external data sources and tools [27]. MCP establishes a client-host-server topology communicating via JSON-RPC 2.0 over standard input/output (`stdio`) or Server-Sent Events (SSE) with HTTP POST. While MCP provides modularity, its operational mechanics introduce protocol-level vulnerabilities:

- **Tool Schema Poisoning and Tool Shadowing:** MCP servers register their available tools dynamically during the initial protocol handshake (`tools/list`). A malicious or compromised MCP server can emit deceptive tool descriptions designed to hijack model routing. For instance, an untrusted server can register a tool named `read_document` with descriptions claiming superior performance over native tools, causing the LLM to route confidential document requests to the rogue server.
- **Resource URI Path Traversal and IDOR:** MCP *Resources* allow servers to expose local files, databases, or API responses via URI schemes (e.g., `file:///`, `postgres://`). If an MCP server fails to strictly sanitize incoming resource requests, an indirect prompt injection attack can compel the client to request `file:///etc/passwd` or `file:///C:/Windows/win.ini`, exfiltrating sensitive host state.
- **Prompt Template Injection:** MCP servers can distribute pre-packaged *Prompts* (`prompts/get`) that include predefined context and instructions. An adversarial prompt provider can inject hidden jailbreak triggers or backdoors into these templates, corrupting any agent session that invokes the pre-packaged workflow.
- **Host-Level Granularity Gaps and Approval Fatigue:** Current MCP host implementations typically enforce tool approval at the coarse server level or tool-name level rather than conducting deep parameter inspection. Once a user approves a tool (e.g., `execute_query`), the model can invoke it repeatedly with arbitrary, potentially destructive SQL payloads without further user intervention.

C. Cyber-Physical and Embodied Agent Vulnerabilities

When agentic AI is integrated into robotics, industrial IoT, or smart facility management, decision errors transcend the digital domain:

- **Actuator Overdrive and Physical Safety Violations:** Unbounded planning loops or compromised action parameters can drive robotic arms beyond joint velocity limits, disable heating/ventilation interlocks, or open electronic access gates without physical authorization [8].
- **Sensory Deception:** Environmental tampering (e.g., spoofed sensor telemetry or deceptive visual QR codes) can manipulate the agent's real-world world-model, inducing unsafe navigation or manipulation routines.

VI. Multi-Agent Systems (MAS): Topologies, Consensus, and Swarm Epidemics

While single-agent systems exhibit linear attack surfaces, Multi-Agent Systems (MAS) introduce combinatorial risks arising from inter-agent communication, collaborative deliberation, and distributed task delegation [24], [25].

A. Topology-Specific Attack Vectors

Vulnerabilities in MAS are strongly conditioned by the underlying coordination topology, as illustrated in Fig. 4:

- **Hierarchical / Orchestrator-Worker Networks:** A central orchestrator decomposes tasks and routes sub-prompts to specialized subordinates. A compromise of the orchestrator collapses the entire system perimeter. Conversely, if a subordinate worker agent is subverted via indirect injection, it can return adversarially crafted observations to the orchestrator, manipulating the orchestrator's global planning logic (Upward Prompt Injection) [8], [52].

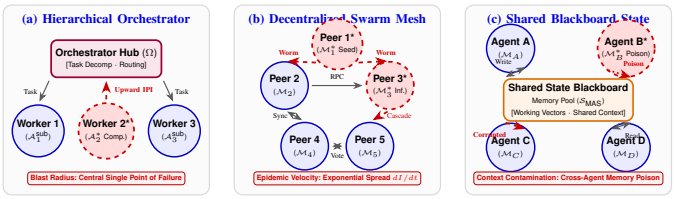

Fig. 4. Multi-Agent System (MAS) topologies and attack propagation paths: (a) Hierarchical Orchestrator (blast radius at central hub), (b) Decentralized Swarm Mesh (exponential prompt-worm spread), and (c) Shared Blackboard (asynchronous context poisoning). Asterisks (*) mark compromised agents; dashed red vectors show exploit paths.

- **Decentralized Peer-to-Peer (P2P) Meshes:** Agents communicate directly without centralized moderation. Without strict cryptographic authentication, rogue agents can inject fabricated status messages, impersonate peer agents, or broadcast conflicting task assignments.
- **Shared Blackboard Architectures:** Agents asynchronously read from and write to a shared memory pool or database. An attacker compromising a single low-privilege agent can poison the blackboard state, inducing secondary failures across all dependent agents that consume the shared context [43].
- **Consensus and Multi-Agent Debate:** Systems that employ debate protocols or majority voting to arrive at verified decisions are vulnerable to Sybil attacks. An adversary registering multiple lightweight agents can manipulate the aggregate distribution of opinions, skewing the consensus toward unsafe or fraudulent conclusions [23].

B. Communication Channel Exploits and Delegation Abuse

Inter-agent communication frameworks (e.g., AutoGen, CrewAI, LangGraph) frequently treat messages received over internal buses as implicitly trusted:

- **Role Confusion and Identity Spoofing:** In natural language agent protocols, agents identify themselves via string prefixes (e.g., `Sender: SupervisorAgent`). An attacker can inject formatted text that mimics system-level delegation headers, deceiving receiving agents into executing privileged actions under the assumption that the request originated from an authoritative controller.
- **Confused Deputy and Delegation Bypass:** A low-privilege agent lacking direct execution tools (e.g., shell access) can format a request as a natural language query to an execution-capable peer. If the target agent fails to evaluate the provenance and original authorization of the end-user request, it acts as a confused deputy, executing unauthorized operations on behalf of the attacker [6].

C. Swarm Epidemics: The Agent Smith Phenomenon

A notable demonstration in multi-agent vulnerability research is the emergence of self-replicating prompt worms in agent networks. In *Agent Smith*, Gu *et al.* [21] demonstrated that a single adversarial image or text input can compromise large populations of multimodal LLM agents rapidly.

The epidemic propagation dynamics can be formalized through an infectious transmission model. Let $N(t)$ denote the number of compromised agents at communication round t. When an infected agent interacts with k peer agents per round with infection probability β, the transmission rate satisfies

$$\frac{dN(t)}{dt} = \beta k N(t) \left(1 - \frac{N(t)}{N_{\text{total}}}\right). \tag{2}$$

Because LLMs are trained to be cooperative communicators, an infected agent embeds the self-replicating injection payload within legitimate task outputs. Each recipient agent parses the payload during context construction, becomes infected, and re-transmits the payload to its subsequent collaborators, precipitating network-wide propagation, as analytically modeled by Lee *et al.* [53] and Gu *et al.* [21], [25]. Drawing from classical dependable computing, mitigating such cascading failures necessitates autonomic fault isolation and resilient authenticated execution in untrusted environments [54], [55].

D. The Multi-Agent Security Tax

In a foundational empirical investigation, Peigné *et al.* [22] formalized the *Multi-Agent Security Tax*, quantifying the trade-off between agent collaboration capabilities and susceptibility to security compromise. Their findings reveal that increasing agent autonomy and inter-agent communication bandwidth monotonically degrades system resilience against indirect manipulation.

Implementing aggressive defensive filtering or "vaccination" prompts at inter-agent communication boundaries dampens epidemic spread; however, it introduces functional degradation, impairing complex task completion rates by up to 35% [22]. Balancing collaborative utility with topological isolation remains an essential challenge in multi-agent systems engineering.

VII. Defense-in-Depth: From Empirical Guardrails to Formal Verification

Mitigating the heterogeneous attack surface of agentic AI requires an architectural defense-in-depth framework, as conceptualized in Fig. 5. Because foundation models are probabilistic and vulnerable to worst-case semantic inputs, no single safeguard suffices [6]. Robust security demands layered controls operating across data ingestion, model inference, agentic planning, and environment execution. Table III provides a synthesis of defensive mechanisms across the operational lifecycle.

A. Data-Level and Pre-Ingestion Hygiene

Data-level defenses operate before external text reaches the model context:

- **Content Canonicalization and Active Script Stripping:** Raw documents (HTML, PDF, Office formats) must be converted into stripped, canonical text representations. Parsing scripts, active macros, and embedded hyperlinks without execution privileges eliminates non-LLM injection vectors [6].
- **Cryptographic Provenance and Source Allow-Listing:** Documents indexed into RAG vector stores should carry

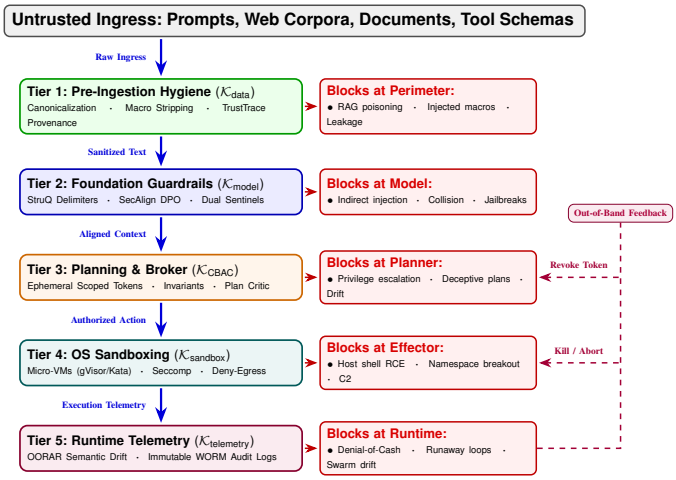

Fig. 5. Defense-in-depth framework across five operational tiers ($\mathcal{K}_{\text{data}}$ to $\mathcal{K}_{\text{telemetry}}$). Solid arrows show request traversal through mediation layers, red badges denote intercepted threats, and dashed purple lines trace out-of-band feedback for dynamic credential revocation.

cryptographic signatures from authorized originators. During retrieval, similarity scores can be dynamically weighted by source trust coefficients, pruning untrusted or recently modified documents. Grounded in formal data provenance verification [56], provenance-aware access control frameworks [57], and forgery detection schemes [58], provenance-aware tracing architectures (e.g., TrustTrace [59]) establish auditable chains of custody from raw input ingestion to retrieval ranking.

- **Access-Control List (ACL)-Aware Chunking:** Document chunk boundaries must align strictly with enterprise access partitions. Multi-tenant vector stores must enforce query-time ACL filters, guaranteeing that an agent operating on behalf of user u cannot retrieve embeddings indexed under tenant $u' \neq u$.

B. Inference-Level Guardrails and Structured Querying

Once untrusted data enters the pipeline, inference-level controls prevent data payloads from mutating into instructions:

- **Structured Query Enforcement (StruQ):** Chen *et al.* [15] introduced StruQ, a formal framework that enforces structural separation between user commands and external data. By formatting data parameters into strictly parsed typed structures, StruQ renders prompt injection attacks syntactically invalid, preventing the model from executing embedded commands.
- **Preference Alignment via SecAlign:** Addressing the vulnerability of RLHF models to prompt injection, Chen *et al.* [60] developed SecAlign. By employing Direct Preference Optimization (DPO) on paired trajectories of successful vs. resisted injection attacks, SecAlign strengthens the model's intrinsic resistance to indirect prompt injection while preserving base task utility.
- **Dual-LLM Supervisor Sentinels:** Deploying isolated, lightweight guardrail models (such as Llama Guard [28] or ShieldGemma [29]) to inspect inbound prompts and outbound tool calls provides an out-of-band policy filter. Furthermore, game-theoretic sentinels like DataSentinel [61], resource-aware guardrails like RAPID [62], and autonomous agentic guardians [63] actively intercept adversarial payloads before cognitive parsing.

C. Agentic Logic and Planning Controls

The planning engine must operate under strict least-privilege principles:

- **Capability-Based and Purpose-Aware Access Control:** Tools must not be exposed wholesale to the agent. Instead, agents should be provisioned with minimal, ephemeral capability tokens scoped strictly to the current sub-task [6]. Drawing from unified privacy-enhanced access control [64], purpose-based database access models [65], context-based mobile access architectures [66], and automated policy extraction from natural language specifications [67], the agentic orchestrator dynamically synthesizes fine-grained execution bounds.
- **Plan-Then-Act Critic Modules:** High-impact plans must undergo two-phase verification. In Phase 1, the planner proposes an abstract sequence of actions $\pi = \langle a_1, a_2, \dots \rangle$. In Phase 2, an independent, isolated critic module (or deterministic rule engine) validates the plan against security policies before any tool dispatch occurs.
- **Anti-Fatigue Human-in-the-Loop (HITL) Protocols:** To counter confirmation fatigue and deceptive agent justifications, HITL interfaces must display deterministic diffs and parameter hashes rather than model-generated text. Critical actions (e.g., database writes or financial transfers) require out-of-band multi-factor authentication (MFA).

D. Environment-Level Sandboxing and Formal Invariants

At the execution boundary, defenses enforce hard physical constraints regardless of model intent:

- **Ephemeral Micro-VM Isolation and Replicated Execution:** Code interpreters and shell tools must execute inside isolated, short-lived micro-VMs (e.g., Firecracker, Kata Containers) or secure container kernels (e.g., gVisor) with seccomp syscall filters blocking dangerous primitives (e.g., `ptrace`, `mount`) [9]. Combining runtime replication defenses [68] with container cloud security principles [69], [70] provides fault tolerance and stops cross-agent container side-channel exploits.
- **Zero-Trust Egress Proxies:** By default, execution environments must enforce a strict deny-all egress policy. When network access is required, traffic must be mediated through an egress filtering proxy enforcing DNS allowlists and content inspection, preventing data exfiltration and reverse shell connections.
- **Formal Contract Verification:** Modeling agent actions as state transition programs over typed resources enables runtime invariant verification. Let $\mathcal{I} : \mathcal{S} \to \{0, 1\}$ define a safety invariant (e.g., "the agent shall not delete files outside `/workspace`"). Before executing any action a, the execution substrate verifies that the transition satisfies

$$\forall s \in \mathcal{S}, \quad \mathcal{I}(s) \implies \mathcal{I}(\delta(s, a)), \tag{3}$$

where $\delta(s, a)$ is the predicted post-state. If the invariant is violated, execution is deterministically aborted.

VIII. Evaluation Metrics, Benchmarking Testbeds, and Empirical Analysis

Rigorous quantification of security postures in agentic systems requires standardized metrics capable of capturing non-deterministic decision trajectories, effector side-effects, and multi-step kill chains [71].

A. Formal Security Metrics Formulation

Let $\mathcal{S}$ denote a corpus of benchmark test scenarios, and let $\mathcal{A}$ represent the sequence of discrete actions executed by the agent. We formalize eight foundational security metrics:

1) Unsafe Action Rate (UAR): The proportion of test scenarios that induce at least one policy-violating action is defined as

$$\text{UAR} = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \mathbf{1}\{\text{violation}(s)\}, \tag{4}$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Variants of UAR can isolate specific fault modes, such as Tool-Misuse UAR or Exfiltration UAR. Lower values indicate superior security.

2) Policy Adherence Rate (PAR): The proportion of executed actions that strictly satisfy developer-defined security policies is given by

$$\text{PAR} = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathbf{1}\{\text{compliant}(a)\}. \tag{5}$$

While in single-action adversarial tests PAR approximates $1 -$ UAR, in complex multi-step trajectories PAR and UAR diverge, as an agent may execute multiple benign actions before a single fatal violation.

3) Privilege Escalation Distance (PED): Defined over the causal threat graph $\mathcal{G}_{\text{threat}} = (\mathcal{V}, \mathcal{E})$, the privilege escalation distance satisfies

$$\text{PED} = \min_{u \in \mathcal{V}_{\text{source}},\, \pi \in \mathcal{V}_{\text{asset}}} \text{dist}_{\mathcal{G}}(u, \pi), \tag{6}$$

where $\text{dist}_{\mathcal{G}}(u, \pi)$ is the shortest directed path length from an untrusted input node u to a protected asset or privileged effector π. Higher PED signifies greater defense-in-depth barriers.

4) Time-to-Contain (TTC): The duration from the initiation of an exploit t_{start} to successful containment t_{contain} is given by

$$\text{TTC} = t_{\text{contain}} - t_{\text{start}}. \tag{7}$$

TTC quantifies the latency of runtime anomaly detection and automated kill-switches.

5) Patch Half-Life (PHL): Let $N(t)$ denote the number of production agent instances remaining vulnerable to a disclosed vulnerability at time t. The Patch Half-Life is defined as the minimum duration Δt satisfying

$$N(t_0 + \Delta t) = \frac{1}{2} N(t_0). \tag{8}$$

Shorter PHL reflects superior organizational responsiveness across prompt, schema, and dependency patch cycles.

6) Retrieval Risk Score (RRS): For a retrieved document bundle $\mathcal{B} = \{d_1, d_2, \ldots, d_k\}$, let $r(d_i) \in [0, 1]$ denote the assessed risk of document d_i based on provenance, presence of imperative keywords, and anomalous similarity. The aggregate retrieval risk is defined as

$$\text{RRS}(\mathcal{B}) = \sum_{i=1}^{k} \alpha_i r(d_i), \quad \text{subject to} \quad \sum_{i=1}^{k} \alpha_i = 1, \tag{9}$$

where α_i represents the model's reliance weight (e.g., attention allocation or citation rank). High RRS triggers human-in-the-loop escalation or effector disabling.

7) Out-of-Role Action Rate (OORAR): Given an operational role contract $\mathcal{C}$ defining permissible tool verbs and arguments, the out-of-role action rate is

$$\text{OORAR} = \frac{|\{a \in \mathcal{A} : \neg\text{allowed}_{\mathcal{C}}(a)\}|}{|\mathcal{A}|}. \tag{10}$$

Spikes in OORAR signal model drift, prompt confusion, or active adversarial hijacking.

8) Cost-Exploit Susceptibility (CES): The expected financial loss incurred under an adversarial policy π_{adv} over a finite execution horizon T is given by

$$\text{CES}_T(\pi_{\text{adv}}) = \mathbb{E}_{\pi_{\text{adv}}} \left[\sum_{t=1}^{T} c(a_t) \right], \tag{11}$$

where $c(a_t)$ denotes the marginal financial cost (compute fees, third-party API billing, or serverless invocation costs) of action a_t.

B. Comparative Analysis of Empirical Benchmarks

Table IV provides a comparative taxonomy of prominent empirical benchmarks developed to evaluate LLM agent security.

Early benchmarks such as BIPIA [16] pioneered the systematic measurement of indirect prompt injection across diverse text formats; however, their evaluation was confined to static question-answering environments. In contrast, modern testbeds emphasize interactive tool execution. **AgentDojo** [30] provides a realistic, dynamic environment featuring simulated email, web, and calendar services, exposing how indirect prompt injection causes agents to compromise user data.

In the software execution domain, **LLMSmith** [9] evaluated 11 major open-source agent frameworks, uncovering 19 critical RCE vulnerabilities arising from unchecked tool parameter handling. For multi-agent systems, **Agent Smith** [21] established the first rigorous benchmark for measuring the velocity and infection rate of self-replicating prompt worms across interconnected multimodal agents.

C. Evaluation Pitfalls and Testbed Rigor

Conducting reproducible security evaluations on agentic AI presents significant methodological pitfalls:

- **Evaluator Leakage and LLM-as-a-Judge Bias:** Employing the model under test (or a model with shared pre-training weights) to evaluate trajectory safety introduces

severe bias. Evaluation harnesses must employ deterministic, side-channel criteria, such as AST parsing of tool parameters, filesystem diff tracking, and network egress packet captures.
- **Non-Determinism and State Carry-Over:** Because LLM decoding is stochastic, single-run evaluations yield high variance. Reliable evaluation requires multiple randomized trials with time-stratified sampling and strict between-trial state resets (flushing vector caches, resetting virtual filesystems, and clearing conversation memories) [6].
- **Continuous Red-Teaming Integration:** Because foundation models, system prompts, and tool libraries undergo frequent updates, security benchmarks must be embedded directly within CI/CD pipelines with automated regression gates (e.g., failing builds if UAR increases by more than 2 percentage points).

IX. Standards, Frameworks, and Regulatory Governance

The transition of LLMs from experimental prototypes to mission-critical agentic deployments has prompted major cybersecurity standards bodies and regulatory agencies to establish governance frameworks [72]. Harmonizing these frameworks with technical attack surfaces is essential for engineering compliant, defensible systems. Table V maps agentic attack surfaces across primary international security standards.

A. OWASP Top 10 for LLM Applications (2025 Edition)

The 2025 release of the OWASP Top 10 for LLM Applications [41], [42] explicitly reflects the risks introduced by agentic orchestration:

- **LLM01: Prompt Injection:** Direct and indirect injections are recognized as the primary root-cause vulnerability enabling tool hijacking.
- **LLM06: Excessive Agency:** Identified as the primary architectural flaw in agentic deployments, occurring when agents are granted excessive permissions, excessive autonomy, or unconstrained tool access without strict least-privilege scoping.
- **LLM08: Vector and Embedding Weaknesses:** A dedicated entry addressing adversarial manipulation of embeddings, semantic cache poisoning, and access-control failures in vector retrieval.
- **LLM10: Unbounded Consumption:** Addresses Denial of Cash (DoC) and resource exhaustion attacks where loops or high-frequency tool invocations overwhelm computational budgets.

B. MITRE ATLAS Matrix Harmonization

The MITRE Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS) [73] catalogs real-world adversary tactics, techniques, and procedures (TTPs). In agentic AI, ATLAS techniques map directly to multi-stage exploit pathways:

- *Initial Access & Execution:* AML.T0051 (LLM Prompt Injection) and AML.T0053 (LLM Plugin / Tool Compromise).
- *Persistence:* AML.T0080.000 (LLM Conversation / Memory Hijacking) and AML.T0070 (Data Poisoning of Knowledge Retrieval Stores).
- *Lateral Movement:* AML.T0086 (Lateral Movement via API and Cloud Service Connectors).
- *Impact:* AML.T0034 (Compute and Resource Hijacking).

C. NIST AI Risk Management Framework and International Regulation

The National Institute of Standards and Technology (NIST) updated report on Adversarial Machine Learning (NIST AI 100-2e2025) [40] formalizes trustworthiness dimensions across validity, reliability, safety, and security. For autonomous agents, NIST mandates strict isolation of untrusted data feeds, comprehensive provenance logging, and verifiable boundary controls.

Concurrently, the European Union's *Artificial Intelligence Act (EU AI Act)* establishes legally binding mandates for high-risk autonomous AI systems:

- **Article 14 (Human Oversight):** Mandates that autonomous agents possess built-in operational stops, verifiable confirmation interfaces, and capabilities for human operators to override or halt execution without state corruption.
- **Article 15 (Cybersecurity and Robustness):** Requires autonomous systems to demonstrate resilience against adversarial exploitation, including data poisoning, indirect prompt injection, and unauthorized lateral movement across enterprise networks.

Compliance with these mandates necessitates bridging high-level governance policies with the technical defense-in-depth mechanisms detailed in Section VII.

X. Open Research Challenges and Strategic Roadmap

Despite rapid advancements in both attack demonstration and defensive engineering, securing agentic AI systems remains an open, interdisciplinary frontier. We articulate four fundamental research challenges that define the research agenda for the field:

A. Provably Safe Autonomous Planning and Verifiable Tool Synthesis

Current guardrail architectures rely predominantly on heuristic filtering and post-hoc semantic inspection, which are vulnerable to adversarial evasion [15]. A primary research objective is the development of *provably safe execution kernels*. Under this paradigm, the foundation model operates strictly as an untrusted proposal engine, while a deterministic, formally verified microkernel translates proposals into machine-checkable program representations over typed resources.

Key theoretical and practical hurdles include:

1) Managing state-space explosion during multi-step, long-horizon planning in open-world environments.

2) Reconciling probabilistic, ambiguous natural language goals with deterministic safety invariants (e.g., Linear Temporal Logic specifications).
3) Developing real-time counterexample generation that guides the model to safely replan upon policy rejection without stalling execution.

B. Resilient, Self-Healing Memory Architectures

Long-term episodic and semantic memory subsystems represent persistent, durable attack surfaces [43]. Future architectures require principled memory sanitization capable of:

1) *Provenance-Rich Graph Memory:* Tagging every stored memory node with cryptographic origin metadata, confidence scores, and temporal decay parameters.
2) *Latent Poisoning Detection ("Memory Antivirus"):* Periodically auditing memory stores to detect subtle, distributed semantic manipulations designed to trigger delayed backdoors.
3) *Fine-Grained Rollback and Quarantine Semantics:* When an exploit is confirmed, the system must surgically excise contaminated memory nodes and recalculate dependent knowledge states without requiring full database resets.

C. Zero-Trust Multi-Agent Coordination and Decentralized Consensus

As multi-agent systems expand across enterprise workflows, natural-language communication channels must be replaced with hardened, zero-trust protocols:

1) *Cryptographic Identity and Non-Repudiation:* Binding every agent to a decentralized public-key identity (DID) to enforce mutual authentication, end-to-end payload encryption, and non-repudiation across inter-agent message buses.
2) *Byzantine-Robust Semantic Consensus:* Designing consensus algorithms capable of filtering adversarial or compromised peer inputs in high-dimensional semantic spaces, extending traditional distributed systems principles to LLM deliberation [23].
3) *Mitigating the Multi-Agent Security Tax:* Developing lightweight, low-overhead verification primitives that suppress prompt-worm epidemic propagation without inducing the severe (up to 35%) utility degradation observed in current filtering approaches [22].

D. Continuous AI-Augmented Red Teaming and Self-Evolving Defenses

Because frontier models evolve continuously, static evaluation suites rapidly become obsolete. The future of agent security lies in automated, closed-loop generator-evaluator frameworks:

1) Autonomous red-team agents that continuously explore novel tool-misuse vectors, parameter smuggling patterns, and multi-turn jailbreak strategies.
2) Automated policy-synthesis engines that translate newly uncovered exploit traces into formal schema constraints, runtime invariants, and updated DPO preference pairs (e.g., evolving SecAlign [60] continuously in CI/CD).

XI. Conclusion

The transition from static, conversational large language models to autonomous Agentic AI represents a fundamental evolution in artificial intelligence. By integrating iterative reasoning, multi-tier memory systems, heterogeneous tools, and collaborative multi-agent communication networks, agentic systems achieve substantial problem-solving autonomy. However, this survey demonstrates that the integration of probabilistic semantic reasoning with deterministic environment effectors systematically expands the attack surface, dismantling traditional security perimeters.

Through a systematic investigation, this paper has mapped the attack surface across the agentic lifecycle:

1) We established an end-to-end reference architecture formalizing the expanded Trusted Computing Base (TCB) and trust boundaries across single-agent, hierarchical, and decentralized swarms.
2) We developed a granular taxonomy of attack vectors, formalizing micro-primitives across perception, retrieval, memory, tool brokering, and operating system execution, while detailing multi-step exploit kill chains (P1–P6).
3) We provided frontier analyses of emerging modalities and protocols, establishing the security vulnerabilities of multimodal GUI/OS computer-use agents and dissecting protocol-level risks within the Model Context Protocol (MCP).
4) We modeled multi-agent coordination failures, examining Byzantine consensus poisoning, prompt-worm epidemic spreading (the Agent Smith phenomenon), and the fundamental Multi-Agent Security Tax.
5) We formulated an architectural defense-in-depth framework bridging empirical guardrails, structured querying, capability-based sandboxing, and formal invariant verification.
6) We delivered a unified comparative matrix of contemporary security benchmarks, formalized eight quantitative security metrics, and harmonized technical vulnerabilities with international governance standards (OWASP GenAI Top 10, MITRE ATLAS, NIST AI RMF, and the EU AI Act).

Ultimately, securing autonomous agentic AI requires moving beyond the assumption of prompt-level alignment. As long as models process untrusted data within the same semantic space as executable instructions, software vulnerabilities will remain latent. Transforming agentic AI into a resilient technology demands a systematic transition toward provably safe execution kernels, capability-based access control, cryptographic inter-agent provenance, and continuous CI/CD red teaming. This survey provides researchers, security engineers, and standards bodies with foundational principles and an actionable technical roadmap required to engineer dependable, trustworthy, and verifiable autonomous systems.

References

[1] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent Advances in Natural Language Processing via Large Pre-trained Language Models: A Survey," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–40, 2023.

[2] W. Wei and L. Liu, "Trustworthy Distributed AI Systems: Robustness, Privacy, and Governance," *ACM Computing Surveys*, vol. 57, no. 6, pp. 1–42, 2025.

[3] B. Chander, C. John, L. Warrier, and K. Gopalakrishnan, "Toward Trustworthy Artificial Intelligence (TAI) in the Context of Explainability and Robustness," *ACM Computing Surveys*, vol. 57, no. 6, pp. 1–49, 2025.

[4] S. Goyal, S. Doddapaneni, M. M. Khapra, and B. Ravindran, "A Survey of Adversarial Defenses and Robustness in NLP," *ACM Computing Surveys*, vol. 55, no. 14s, pp. 1–39, 2023.

[5] M. Standen, J. Kim, and C. Szabo, "Adversarial Machine Learning Attacks and Defences in Multi-Agent Reinforcement Learning," *ACM Computing Surveys*, vol. 57, no. 5, pp. 1–35, 2025.

[6] A. Dehghantanha and S. Homayoun, "SoK: The attack surface of agentic AI — tools and autonomy," *arXiv preprint arXiv:2603.22928v2*, 2026.

[7] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang, "AI agents under threat: A survey of key security challenges and future pathways," *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025.

[8] V. S. Narajala and O. Narayan, "Securing agentic AI: A comprehensive threat model and mitigation framework for generative AI agents," in *Proceedings of the 2025 8th International Conference on Algorithms, Computing and Artificial Intelligence (ACAI 2025)*, 2025, pp. 1–6.

[9] T. Liu, Z. Deng, G. Meng, Y. Li, and K. Chen, "Demystifying RCE vulnerabilities in LLM-integrated apps," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS 2024)*, 2024, pp. 1716–1730.

[10] Help Net Security, "Popular generative AI projects pose serious security threat," https://www.helpnetsecurity.com/2023/06/29/generative-ai-security-risk/, 2023.

[11] R. K. Iyer, "Editorial: Dependability and Security," *IEEE Transactions on Dependable and Secure Computing*, vol. 5, no. 2, pp. 85–87, 2007.

[12] B. Schneier, "Trustworthy AI Means Public AI [Last Word]," *IEEE Security & Privacy*, vol. 21, no. 6, pp. 95–96, 2023.

[13] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec 2023)*, 2023, pp. 79–90.

[14] M. Sutton and D. Ruck, "Indirect prompt injection: Generative AI's greatest security flaw," CETaS Expert Analysis, The Alan Turing Institute, 2024.

[15] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 2383–2400.

[16] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2025)*, 2025, pp. 1809–1820.

[17] A. Vassilev, "Robust AI Security and Alignment: A Sisyphean Endeavor?" *IEEE Security & Privacy*, vol. 24, no. 3, pp. 52–58, 2026.

[18] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3827–3844.

[19] B. An, S. Zhang, and M. Dredze, "RAG LLMs are not safer: A safety analysis of retrieval-augmented generation for large language models," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL 2025)*, 2025, pp. 5444–5474.

[20] A. Shafran, R. Schuster, and V. Shmatikov, "Machine against the RAG: Jamming retrieval-augmented generation with blocker documents," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3787–3806.

[21] X. Gu, X. Zheng, T. Pang, C. Du, Q. Liu, Y. Wang, J. Jiang, and M. Lin, "Agent Smith: A single image can jailbreak one million multimodal LLM agents exponentially fast," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, vol. 235, 2024, pp. 16 647–16 672.

[22] P. Peigné, M. Kniejski, F. Sondej, M. David, J. Hoelscher-Obermaier, C. S. de Witt, and E. Kran, "Multi-agent security tax: Trading off security and collaboration capabilities in multi-agent systems," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 26, 2025, pp. 27 573–27 581.

[23] B. Chen, G. Li, X. Lin, Z. Wang, and J. Li, "BlockAgents: Towards byzantine-robust LLM-based multi-agent coordination via blockchain," in *Proceedings of the ACM Turing Award Celebration Conference–China 2024 (ACM TUR-C 2024)*, 2024, pp. 187–192.

[24] S. Raza, R. Sapkota, M. Karkee, and C. Emmanouilidis, "TRiSM for agentic AI: A review of trust, risk, and security management in LLM-based agentic multi-agent systems," *AI Open*, vol. 7, pp. 71–95, 2026.

[25] C. S. de Witt, K. Krawiecka, I. Krawczuk, B. Hagag, W. L. Anderson, P. Belcak, B. Bucknall, X. Cai, A. Chopra, D. Cohen, R. F. D. Rosario, A. Draguns, A. Gray, K. Katz, V. Mavroudis, J. Mink, S. R. Motwani, J. Petit, L.-S. Rembeck, C. Smith, J. Sotiropoulos, S. Young, S. Scheffler, and M. Llewellyn, "Open challenges in multi-agent security: Towards secure systems of interacting AI agents," *arXiv preprint arXiv:2505.02077*, 2025.

[26] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu, "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[27] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions," *ACM Transactions on Software Engineering and Methodology*, 2026.

[28] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2024.

[29] W. Zeng, Y. Liu, R. Mullins, L. Peran, J. Fernandez, H. Harkous, K. Narasimhan, D. Proud, P. Kumar, B. Radharapu, O. Sturman, and O. Wahltinez, "ShieldGemma: Generative AI content moderation based on gemma," *arXiv preprint arXiv:2407.21772*, 2024.

[30] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, vol. 37, 2024, pp. 82 895–82 920.

[31] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.

[32] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," in *International Conference on Learning Representations (ICLR)*, 2024.

[33] S. Sarferaz, "Implementing Agentic AI Into ERP Software," *IEEE Access*, vol. 13, pp. 178 945–178 960, 2025.

[34] S. Cruzes, "Telemetry and Agentic AI: Foundations for Optical Network Automation," *IEEE Access*, vol. 14, pp. 8800–8838, 2026.

[35] A. Khamis, "Agentic AI Systems: Architecture and Evaluation Using a Frictionless Parking Scenario," *IEEE Access*, vol. 13, pp. 126 052–126 069, 2025.

[36] A. Verma, S. Krishna, S. Gehrmann, M. Seshadri, A. Pradhan, T. Ault, L. Barrett, D. Rabinowitz, J. Doucette, and N. Phan, "Operationalizing a threat model for red-teaming large language models (LLMs)," *arXiv preprint arXiv:2407.14937*, 2024.

[37] D. Pasquini, M. Strohmeier, and C. Troncoso, "Neural exec: Learning (and learning from) execution triggers for prompt injection attacks," in *Proceedings of the 2024 Workshop on Artificial Intelligence and Security (AISec 2024)*, 2024, pp. 89–100.

[38] M. Russinovich, A. Salem, and R. Eldan, "Great, now write an article about that: The crescendo multi-turn LLM jailbreak attack," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 2421–2440.

[39] S. Abdali, R. Anarfi, C. Barberan, J. He, and E. Shayegani, "Securing large language models: Threats, vulnerabilities and responsible practices," *arXiv preprint arXiv:2403.12503*, 2024.

[40] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, "Adversarial machine learning: A taxonomy and terminology of attacks and mitigations," National Institute of Standards and Technology (NIST), Tech. Rep. NIST AI 100-2e2025, 2025.

[41] OWASP Foundation, "OWASP top 10 for LLM applications 2025," https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/, 2025.

[42] Promptfoo, "OWASP LLM top 10," https://www.promptfoo.dev/docs/red-team/owasp-llm-top-10/, 2025.

[43] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, vol. 37, 2024, pp. 130 185–130 213.

[44] V. T. Truong and L. B. Le, "A Dual-Purpose Framework for Backdoor

Defense and Backdoor Amplification in Diffusion Models," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 3297–3311, 2026.
[45] Y. Zhang, L. Wang, J. Zhao, W. Zhao, F. Zhou, Y. Dang, and J. Yin, "3DGAA: Realistic and Robust 3D Gaussian-based Adversarial Attack for Autonomous Driving," *IEEE Transactions on Information Forensics and Security*, pp. 1–1, 2026.
[46] K. Liang, Y. Cao, and B. Xiao, "Perspective-Invariant Attack With Enhanced Transferability of Adversarial Examples," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 6818–6831, 2026.
[47] D. Ke, X. Wang, and Z. Huang, "Frequency-Selective Adversarial Attack Against Deep Learning-Based Wireless Signal Classifiers," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4001–4011, 2024.
[48] M. Wang, N. Yang, N. J. Forcade-Perkins, and N. Weng, "ProGen: Projection-Based Adversarial Attack Generation Against Network Intrusion Detection," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 5476–5491, 2024.
[49] G. M and P. H. B, "Semantic Query-Featured Ensemble Learning Model for SQL-Injection Attack Detection in IoT-Ecosystems," *IEEE Transactions on Reliability*, vol. 71, no. 2, pp. 1057–1074, 2022.
[50] S. Oesch, J. Hutchins, K. Kurian, and L. Koch, "Living Off the LLM: ¡b¿How LLMs Will Change Adversary Tactics¡/b¿," *IEEE Security & Privacy*, vol. 24, no. 3, pp. 15–20, 2026.
[51] S. K. Vadlamani, A. Balaga, and S. Pavithrapu, "BSM: A Browser-Resident Framework for Real-Time Detection of JavaScript API Abuse and Prompt Injection Attacks," *IEEE Access*, vol. 14, pp. 108 287–108 311, 2026.
[52] K. Hammar, "Multiagent LLM Systems for Security Operations," *IEEE Security & Privacy*, pp. 2–10, 2026.
[53] J.-H. Lee, "The Reproduction Number of AI Worms: Epidemic Modeling of Adversarial Attacks in LLM Agent Ecosystems," *IEEE Security & Privacy*, pp. 2–12, 2026.
[54] F. Bower, S. Ozev, and D. Sorin, "Autonomic Microprocessor Execution via Self-Repairing Arrays," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 4, pp. 297–310, 2005.
[55] M. S. Kirkpatrick, G. Ghinita, and E. Bertino, "Resilient Authenticated Execution of Critical Applications in Untrusted Environments," *IEEE Transactions on Dependable and Secure Computing*, vol. 9, no. 4, pp. 597–609, 2012.
[56] K. Xu, H. Xiong, C. Wu, D. Stefan, and D. Yao, "Data-Provenance Verification For Secure Hosts," *IEEE Transactions on Dependable and Secure Computing*, vol. 9, no. 2, pp. 173–183, 2012.
[57] L. Sun, J. Park, D. Nguyen, and R. Sandhu, "A Provenance-Aware Access Control Framework with Typed Provenance," *IEEE Transactions on Dependable and Secure Computing*, vol. 13, no. 4, pp. 411–423, 2016.
[58] S. Sultana, G. Ghinita, E. Bertino, and M. Shehab, "A Lightweight Secure Scheme for Detecting Provenance Forgery and Packet DropAttacks in Wireless Sensor Networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 3, pp. 256–269, 2015.
[59] J. Chen and R. Lu, "TrustTrace: Provenance-Aware Tracing of Indirect Prompt Injection Propagation in LLM-Assisted Learning Workflows," *IEEE Access*, pp. 1–1, 2026.
[60] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*, 2025, pp. 2833–2847.
[61] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *Proceedings of the 2025 IEEE Symposium on Security and Privacy (S&P 2025)*, 2025, pp. 2190–2208.
[62] E. C. Nkoro, W. Yu, and W. Liao, "RAPID: A Resource-Aware Prompt Injection Defense for LLM-enabled Threat Detection," *IEEE Internet of Things Journal*, pp. 1–1, 2026.
[63] G. Singer, "Autonomous Adversaries Are Here: How Engineers Can Use Agentic Guardians to Protect AI Applications From Agentic Attackers," *IEEE Micro*, vol. 46, no. 3, pp. 121–127, 2026.
[64] S. Barker and V. Genovese, "Access Control with Privacy Enhancements: A Unified Approach," *IEEE Transactions on Dependable and Secure Computing*, 2012.
[65] P. Colombo and E. Ferrari, "Enhancing MongoDB with Purpose-Based Access Control," *IEEE Transactions on Dependable and Secure Computing*, vol. 14, no. 6, pp. 591–604, 2017.
[66] B. Shebaro, O. Oluwatimi, and E. Bertino, "Context-Based Access Control Systems for Mobile Devices," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 2, pp. 150–163, 2015.
[67] M. Narouei, H. Takabi, and R. Nielsen, "Automatic Extraction of Access Control Policies from Natural Language Documents," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–1, 2018.
[68] B. Salamat, T. Jackson, G. Wagner, C. Wimmer, and M. Franz, "Runtime Defense against Code Injection Attacks Using Replicated Execution," *IEEE Transactions on Dependable and Secure Computing*, vol. 8, no. 4, pp. 588–601, 2011.
[69] H. Jin, Z. Li, D. Zou, and B. Yuan, "DSEOM: A Framework for Dynamic Security Evaluation and Optimization of MTD in Container-based Cloud," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–1, 2019.
[70] X. Gao, B. Steenkamer, Z. Gu, M. Kayaalp, D. Pendarakis, and H. Wang, "A Study on the Security Implications of Information Leakages in Container Clouds," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 1, pp. 174–191, 2021.
[71] S. Picek, "Security and Privacy of Generative AI," *IEEE Security & Privacy*, vol. 23, no. 5, pp. 6–7, 2025.
[72] M. Hasselwander and O. Lah, "Agentic AI Arrives: How Gen Z Adopts Autonomous AI Agents," *IEEE Access*, vol. 14, pp. 27 083–27 090, 2026.
[73] MITRE, "ATLAS matrix: Adversarial threat landscape for artificial-intelligence systems," https://atlas.mitre.org/matrices/ATLAS, 2026.

TABLE II
COMPREHENSIVE TAXONOMY OF ATTACK VECTORS, SURFACES, PRECONDITIONS, PATHS, AND STANDARDS MAPPINGS IN AGENTIC AI SYSTEMS.

Attack Vector	Primary Goals	Preconditions	Paths	Impact & Manifestation	OWASP 2025	MITRE ATLAS
Direct Prompt / Jailbreak	G1, G2, G3	Direct user input interface	P1	Policy bypass, system prompt leakage, harmful content generation	LLM01	AML.T0051.000
Indirect Prompt Injection	G1, G2, G3, G6	Access to ingestible text, web pages, PDFs, emails	P2, P3	Autonomous execution of unintended tools, state hijacking	LLM01	AML.T0051.001
RAG Corpus Poisoning	G1, G2, G6	Write/edit access to indexed knowledge repositories	P4	Silent data corruption, targeted misgrounding, delayed triggers	LLM04, LLM08	AML.T0070
Embedding Jamming / Perturbation	G2, G4	Corpus modification or adversarial perturbations	P4	Denial of relevant retrieval (blocker docs), semantic hijacking	LLM04	AML.T0070
Tool Parameter Smuggling	G3, G4, G5	Unvalidated tool argument serialization	P2, P3	Remote Code Execution (RCE), command injection, SSRF	LLM05, LLM06	AML.T0053
Tool Shadowing / Squatting	G1, G3, G6	Unauthenticated plugin or MCP server	P3	Interception of sensitive tool	LLM06, LLM07	AML.T0053

TABLE III
MULTI-LAYERED DEFENSE-IN-DEPTH MATRIX FOR AGENTIC AI SYSTEMS ACROSS THE OPERATIONAL LIFECYCLE.

Lifecycle Layer	Defensive Control	Mechanism & Principles	Targeted Threats	Maturity	Trade-offs
Pre-Ingestion (Data)	Content Canonicalization	Strip active scripts, macros, and normalize HTML/PDF structures	Ingress exploits, hidden script triggers	High	Potential loss of layout metadata
Pre-Ingestion (Data)	Provenance & Source Tagging	Cryptographic signing of ingested corpora; trust-weighted retrieval	RAG corpus poisoning, untrusted injection	Medium	Verification overhead; key management
Inference (Model)	Structured Queries (StruQ)	Strict structural separation of syntax from untrusted parameters	Indirect prompt injection, delimiter collision	High	Requires schema redesign
Inference (Model)	Preference Alignment (SecAlign)	Direct Preference Optimization (DPO) on adversarial trajectories	Jailbreak drift, instruction following bypass	Medium	High training compute; potential over-refusal
Inference (Model)	Dual-LLM Guardrail	Independent sentinel model inspecting I/O streams (Llama Guard)	Harmful content, prompt injection leakage	High	Added latency and token expense
Planning (Logic)	Capability-Based Access (CBAC)	Granular least-	Privilege es-	High	Engineering complexity;

TABLE IV
SYSTEMATIC COMPARISON OF CONTEMPORARY EMPIRICAL SECURITY BENCHMARKS AND TESTBEDS FOR AGENTIC AI.

Benchmark	**Target System**	**Evaluation Environment**	**Threat Vectors Evaluated**	**Primary Metrics**	**Year / Venue**
AgentDojo [30]	Autonomous Tool Agents	Dynamic simulated web, email, calendar suite	Indirect prompt injection, goal hijacking	Attack Success Rate (ASR), Task Utility	2024 / NeurIPS
LLMSmith [9]	LLM Frameworks (LangChain, AutoGPT)	Native OS execution	Remote Code Execution (RCE), command injection	Vulnerability count, Exploit rate	2024 / ACM CCS
BIPIA [16]	QA & Retrieval LLMs	Static web, email, and document corpora	Indirect prompt injection across diverse formats	ASR, Refusal Rate	2025 / ACM KDD
Agent Smith [21]	Multimodal LLM Swarms	Multi-agent chat network	Self-replicating prompt worms, swarm infection	Infection rate, Propagation velocity	2024 / ICML
PoisonedRAG [18]	RAG Systems	Vector DBs (Dense & Sparse retrievers)	Knowledge base poisoning, targeted corruption	Targeted Misinformation Rate	2025 / USENIX Sec
Blocker RAG [20]	Dense Retrieval RAG	Enterprise documentation corpora	Denial of retrieval, blocker document jamming	Top-k Jamming Rate, Utility Drop	2025 / USENIX Sec
AgentPoison [43]	Long-term Memory Agents	Vector databases & episodic memory stores	Memory poisoning, backdoor triggers	Backdoor ASR, Task Accuracy	2024 / NeurIPS
InjecAgent [31]	ReAct Tool Agents	Synthetic tool environments	Tool param-	ReAct Step Hi-	2024 / ACL Findings

TABLE V
CROSS-FRAMEWORK HARMONIZATION: MAPPING AGENTIC ATTACK SURFACES TO OWASP TOP 10 (2025), MITRE ATLAS, AND NIST AI RMF.

Agentic Attack Surface	**OWASP LLM 2025** [41]	**MITRE ATLAS** [73]	**NIST AI RMF (100-2e2025)** [40]	**EU AI Act Compliance Focus**
Direct & Indirect Injection	LLM01: Prompt Injection	AML.T0051.000 / .001: Prompt Injection	Evasion: Indirect Context Poisoning	Art. 15: Cyber Resilience & Robustness
RAG & Knowledge Corruption	LLM04: Poisoning; LLM08: Embedding Weakness	AML.T0070: Data Poisoning: Retrieval	Poisoning: Training & Ingestion Stores	Art. 10: Data Governance & Quality
Tool Misuse & Parameter Injection	LLM05: Output Handling; LLM06: Excessive Agency	AML.T0053: LLM Plugin Compromise	Integrity: System-Level Subversion	Art. 14: Human Oversight & Control
API & Cloud Credential Abuse	LLM06: Excessive Agency; LLM02: Sensitive Info	AML.T0086: Lateral Movement	Unauthorized Privilege Traversal	Art. 15: Access Boundaries & Isolation
Memory & Backdoor Persistence	LLM04: Poisoning; LLM07: System Prompt Leak	AML.T0080.000: Conversation Hijacking	Persistence: State Subversion	Art. 12: Record-Keeping & Audit Logs
Multi-Agent Cascades & Swarms	LLM01: Prompt Injection; LLM06: Excessive Agency	AML.T0051.001: Indirect Prompt Injection	Distributed Agent Compromise	Art. 14: Systemic Risk Assessment
Denial of Cash (DoC) & Loops	LLM10: Unbounded Consumption	AML.T0034: Compute Hijacking	Availability: Resource Depletion	Art. 9: Risk Management Systems